Flat Containers

- P0038R0
- LEWG, SG14
- Date: 2015-09-25

- Reply-to: Sean Middleditch sean@seanmiddleditch.com

Introduction

This paper serves to explain the desire for the utility of the *flat associative containers* (`flat_map`, `flat_set`, `flat_multimap`, and `flat_multiset`), named after their Boost implementation.

These containers are associative containers (maps and sets) which use contiguous storage, as compared to the standard `std::map` and `std::set` which are node-based containers.

Purpose

One of the primary desires of the flat containers is that they provide a contiguous memory space for element storage. This provides several benefits, such as better cache access pattern and greatly reduced number of calls into the allocator. Some performance numbers are available in the article Traversing a linearized tree by Joaquín M López Muñoz.

The standard currently offers only node-based ordered associative containers. While these data structures have their strengths and uses and offer reasonable performance for general purpose applications, there are some domains (games, real-time, and embedded, among others) that are not always well served by the properties of node-based containers. There is a need for associative containers that maintain good cache locality for both find and iteration and minimize interaction with memory allocators. The flat containers covered by this paper are intended to satisfy this need.

Previous Work

Matt Austern's paper Why you shouldn't use set (and what you should use instead)

Andrei Alexandrescu described such a structure in Modern C++ Design and implemented it in Loki as AssocVector

Boost includes flat containers in their containers module.

The author of this paper has experience implementing and using similar data structures.

Need For a New Library

While an *unordered container* will generally have far faster lookup and insert time than a flat container, the unordered containers have the disadvantage of being unusable with current standard algorithms like `std::set_union` or `std::set_intersection` which require in-order iteration of elements. Flat containers are ordered, like `std::map` or `std::set`, and so the flat containers are usable with algorithms that require in-order element traversal.

A possible implementation of a flat container is to store elements in a sorted array. This implementation allows `O(log(N))` lookup time; insertion and erasure time is `O(N)` in theory, though the practical performance of such operations with a flat container with trivially-movable types for small-ish `N` competes well with the `O(log(N))` time of the node-based containers.

C++ today does not offer good facilities for maintaining a data structure such as these. Assuming that the structure is implemented as a sorted vector, the only available tool offered by the C++ library today is `std::lower_bound`. However, this algorithm does not directly find the value requested, requiring extra checks for use in find or erase operations. For insert, additional checks are also required to avoid inserting duplicate elements.

These extra checks are not particularly onerous, but the author has seen mistakes arise in their use of the years (such as an algorithm that stated it would only insert unique elements but allowed duplicates).

Simply using algorithms to access an existing container does not enforce the invariants of the containers on other generic algorithms using said container. For instance, if users want a particular `vector` to contain only sorted elements, algorithms can easily break that invariant. We can see this today in the number of algorithms that require in-order element traversable (e.g. `std::set_union`) and how they silently fail on unsorted containers.

A custom container or adaptor that enforces these invariants can also provide a similar interface to the current standard's current associative containers, which is important for composibility.

Design Considerations

Key Properties

The flat containers have several key properties that differentiate them from the current standard associative containers. The key difference between the flat containers and the unordered containers is that flat containers are *ordered*.

The key difference between the flat containers and the ordered standard containers is that iterator stability is not guaranteed in the flat containers after insert or erase operations. This is an artifact of not using node-based storage for elements.

Open Design Questions

Adaptor or Container

Should the flat containers be actual containers or should they be container adaptors?

The most natural representation for many uses of a flat container would be a single contiguous array with exponential growth properties, similar to `std::vector`. Implementations like Boost's are true containers, backed internally by a vector-like storage. The user has no option to use another backing store. In the author's experience, this has never been a real problem for use in real-time interactive simulations, which often have particular memory allocation and layout concerns.

However, some users may find that they are better off with a segmented allocation model similar to `std::deque`. Yet other users may have specialized or custom backing containers that they wish to use, such as a fixed-size or stack-allocated array type. An adapator interface for flat containers would allow users to plug in whichever backing container they wish. This has the benefit of making the flat containers simpler as well, as they can easily be implemented as overloads of `insert`, `erase`, and `find`.

The author has a slight preference for a making actual containers as that aligns closely with his practical experience. Feedback from SG14 indicated some interested in the adaptor approach.

Internal Algorithm

Feedback from SG14 indicated strong interest in a level-order implementation rather than a sorted vector implementation. Interest was also raised for an ability to select the underlying algorithm, or separate containers or algorithms.

The simplest internal algorithm is to store sorted elements. This then allows insert, find, and erase operations to all operate in `O(log(N))` time (binary search), plus the time for shifting elements for insert and erase (typically very small for trivially-movable types, as `memmove`-like operations can be used for shifting).

Another option is to use a heap-like algorithm. This approach improves on the cache locality of the search portion of the find algorithm. The author has no practical experience using this structure for a flat container implementation, but the approach is potentially promising.

In the author's opinion, some more study and analysis is warranted before making a firm design decision on this topic.

Reference Proxies

A map container typically stores keys and values together as a `pair`. Another option is to store the keys separately from the values. This condenses the keys in memory and further improves cache locality, both for sorted vector and heap traversal algorithms (though only significantly for small `N` in the sorted vector).

However, this results in the element type of the container necessarily being a pair of references, which is different than other current standard containers and has some caveats with standard idioms and algorithms. See "To Be or Not Be (an Iterator)" by Eric Niebler about some of the problems with containers (and their iterators) that return reference proxies instead of real references.

The author's opinion is that this design point is dependent both on the chosen internal algorithm as well as whether the standard library proxy iterator support.

Exception Guarantees

It is not possible to offer strong exception guarantees with the flat containers, but we can offer the basic exception guarantee. The question is whether this is acceptable.

We can see the problem preventing the strong guarantee by looking at the base algorithms and the necessary invariants.

When inserting an element into the middle of a vector which must stay sorted and contain only unique elements, and in which the vector's capacity already exceeds its size, elements must be shifted to the right to make room for the new element. If these elements can throw on move/copy, it is possible for an exception to be thrown partway through shifting the elements. At this point, the invariants of the data (sorted and unique elements) will not be consistent. However, making the invariant hold either requires rewinding the moves/copies that already took place (which could throw) or completing the shift and copying in the new element (which could throw).

A simple visualation follows. The numbers are abstractions that represent the 'sort key' of complicated objects stored in a vector.

```
// v is a vector of objects which must remain sorted and unique
v = {1, 3, 4, 5}

// v has excess capacity, so no new memory block needs to be allocated
// notably, this means that we must shift elements and mutate v in place
assert(v.capacity() > v.size());

// insert a new element, which should go between objects 1 and 3
v.insert(2)
```

Lets look at what the insert operation must do. It must shift elements right to make room, then finally copy 2 into place.

```
   {1, 3, 4, 5, -}
a. {1, 3, 4, 5, 5} // copy construct 5 into the new final location
b. {1, 3, 4, 4, 5} // copy assign 4 to the old 5
c. {1, 3, 3, 4, 5} // copy assign 3 to the old 4
d. {1, 2, 3, 4, 5} // copy assign 2 to the old 3
```

The problem can arise if we throw during step c. At this point, the invariant of unique elements are already violated (4 is duplicated). Undoing the operations requires shifting elements back to the left, which involves more copy operations, which themselves can throw.

Completing the insert operation is the other obvious way to maintain the invariant, but that also requires yet more copy operations which can throw.

A remaining option is to simply clear the container on exception. This maintains the invariant as the empty set is both ordered and contains no duplicate elements. However, the author dislikes this solution as it results in lost data which may be critical to the user.

Using only types with noexcept move/copy also bypasses the problems entirely. This is the author's primary usage experience, given the frequent use of the `-fno-exceptions` (or equivalent) compiler flag in the games industry, which may explain why the exception guarantees of the flat containers haven't been an actual concern for many of its users.

The author's opinion is that the flat containers can provide only a basic exception guarantee without problem, as that models existing practice.

Delayed Insert/Sort

A potential interface addition over the standard associative continers is to allow a "delayed insert." Such an operation allows the user to amortize the cost of the insertion sort operation when multiple elements are being inserted into the container.

This is different than inserting a range of values as the inserted elements may not be sourced from another container, precluding the existance of any range of iterators to insert from. An example might be a file parsing interface that produces elements (not using input iterators) from a user-supplied file.

Such an interface would require two different additional operations: the delayed insertion operation and a "finalize insertion" operation. An implementation could insert delayed elements in an unsorted manner to extra storage space in the container and the finalize insertion operation can then sort those elements into the live range of the container.

Some implementation perform the insert finalization upon iteration. This causes begin/end operations to mutate the container, however, which can be surprising and problematic for concurrent access.

The author has implemented this feature in a production codebase but it saw very little use.

The author's opinion is that this support is useful to provide though not essential.

References

- Recent SG14 Reflector Discussions

- https://groups.google.com/a/isocpp.org/forum/#!topic/sg14/WyYpdrWxj9Q
- https://groups.google.com/a/isocpp.org/forum/#!topic/sg14/csLjKbLOArw
- Prior Art
- Loki AssocVector - Alexandrescu
- Boost flat containers
- “Why you shouldn’t use set (and what you should use instead)” - Matt Austern
- “Cache-friendly binary search” - Joaquín M López Muñoz
- “Traversing a linearized tree” - Joaquín M López Muñoz
- Contributors & Thanks
- SG14
- J F Bastien
- Matthew Bentley
- Lawrence Crowl
- Brian Ehlert
- Brent Friedman
- Ion Gaztañaga
- Nicolas Guillemot
- Joël Lamotte
- Nevin Liber
- Guillaume Matte
- John McFarlane
- Rene Rivera
- Patrice Roy
- Ville Voutilainen
- Chuck Walbourn
- Scott Wardle
- Michael Wong